INSTITUTO TECNOLÓGICO DE MÉXICO

CAMPUS ORIZABA

“Actividad 3”

EQUIPO 9

GARCIA CEDEÑO IRVING EMANUEL

ROJAS BONILLA SANTIAGO

MARTINEZ ROSAS JOSE ANGEL

MATERIA: PROGRAMACION ORIENTADA A OBJETOS

GRUPO: VESPERTINO 1-2 PM

MAESTRO: RAFAEL HERRERA GARCIA

FECHA DE ENTREGA: 6 DE MARZO DE 2026

Objetivo: Construir clases en Java partiendo de diagramas UML que se obtienen de problemas propuestos

Problema 1:

Cierto músico está haciendo una clasificación de sus composiciones, de cada composición, registra su título, duración de la pista en minutos y segundos, los intérpretes que le han solicitado interpretarla, género musical, fecha de registro, fecha de estreno. Considerar que cada vez que un músico le solicita interpretar alguna de sus composiciones se debe agregar el nombre del músico a los intérpretes. Incluso el intérprete puede ser el mismo autor. Los datos que se tienen de cada solicitud autorizada y registrada son, nombre del intérprete, fecha de solicitud, fecha de autorización que siempre es la misma o fecha posterior a la solicitud, compañía con la que grabará dicha melodía.

Análisis del problema: el problema nos pide tener una clasificación de composiciones, teniendo registrado su título, duración de la pista en minutos y segundos, los intérpretes, y que se puede registrar el mismo autor, nos pide los datos de cada solicitud solicitada y autorizada el nombre del interprete, fecha de solicitud, fecha de autorización que debe ser la misma en la que se grabara la melodía.

Clase:

Composiciones			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
titulo	String	1	si
duracionSegundos	int	1	si
interprete	String	1	si
solicitudes	Solicitudes	1	si
generoMusical	String	1	si
fechaRegistro	LocalDate	1	si
fchaEstreno	LocalDate	1	si

Operaciones composiciones:

Composiciones			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
agregarInterprete	void	Nombre interprete	public
registrarSolicitud	void	solicitud	public
mostrarInformacion	void		public

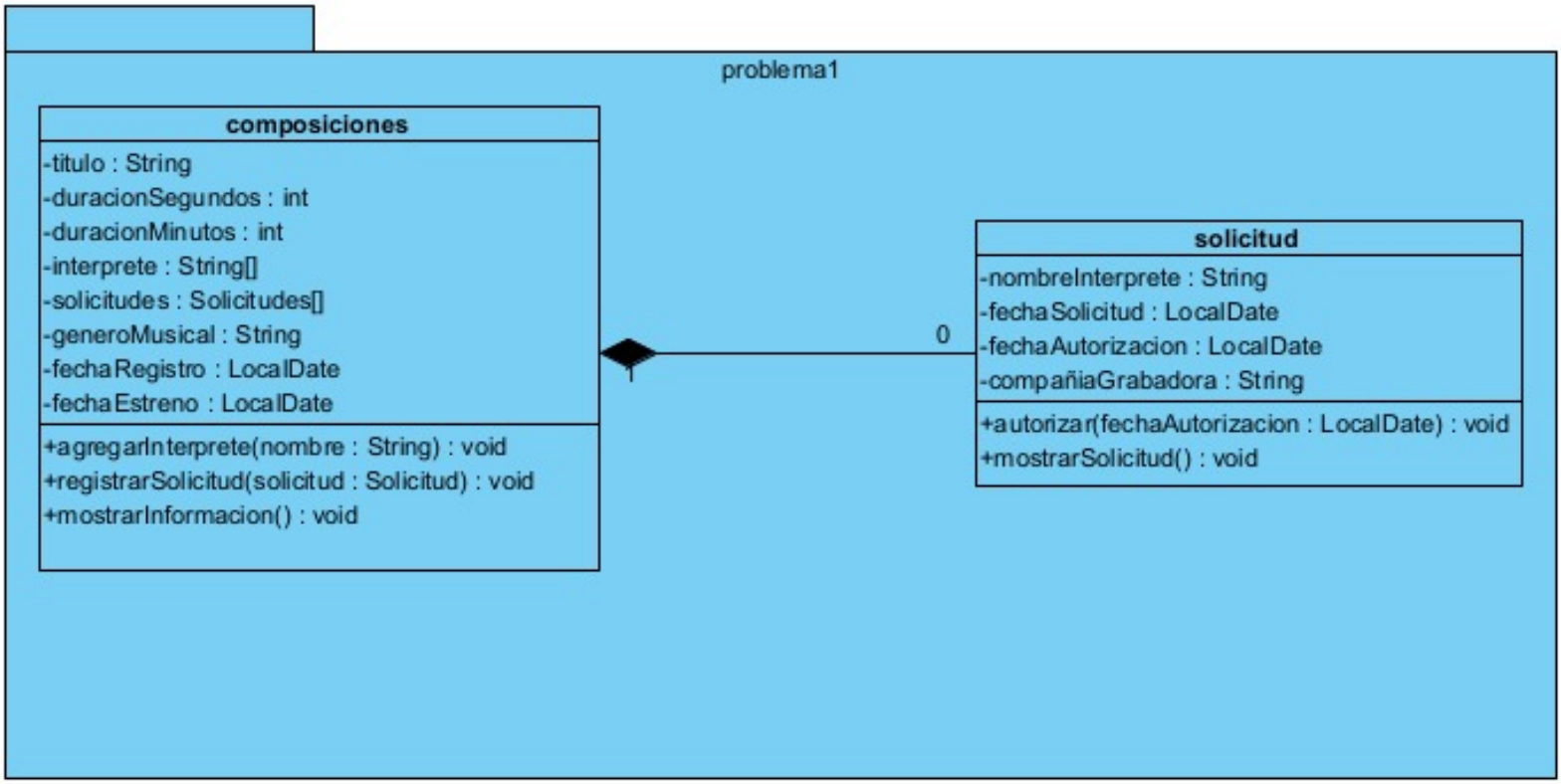
Clase Solicitud:

Solicitud			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
nombreInterprete	String	1	si
fechaSolicitud	LocalDate	1	si
fechaAutorizacion	LocalDate	1	si
compaliaGrabadora	String	1	si

Operaciones solicitud:

Composiciones			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
autorizar	void	fechaAturizacion	public
mostrarSolicitud	void		public

Diagrama:



Código composiciones:

```
Composicion.java ×   Solicitud.java
package problema1;

import java.time.LocalDate;

public class Composicion {  no usages  new *
        private String titulo;  2 usages
        private int duracionSegundos;  2 usages
        private int duracionMinutos;  2 usages
        private String[] interprete;  4 usages
        private Solicitud[] solicitudes;   // CORREGIDO  4 usages
        private String generoMusical;  2 usages
        private LocalDate fechaRegistro;  2 usages
        private LocalDate fechaEstreno;  2 usages

        public Composicion(String titulo, int duracionMinutos, int duracionSegundos,  no usages  new *
                           String generoMusical, LocalDate fechaRegistro, LocalDate fechaEstreno) {

            this.titulo = titulo;
            this.duracionMinutos = duracionMinutos;
            this.duracionSegundos = duracionSegundos;
            this.generoMusical = generoMusical;
            this.fechaRegistro = fechaRegistro;
            this.fechaEstreno = fechaEstreno;

            interprete = new String[10];
            solicitudes = new Solicitud[10];
        }
```

Composicion.java × Solicitud.java

```java
public class Composicion {  no usages  new *

        public void agregarInterprete(String nombre) {  no usages  new *
            for (int i = 0; i < interprete.length; i++) {
                if (interprete[i] == null) {
                    interprete[i] = nombre;
                    break;
                }
            }
        }

        public void registrarSolicitud(Solicitud solicitud) {  no usages  new *
            for (int i = 0; i < solicitudes.length; i++) {
                if (solicitudes[i] == null) {
                    solicitudes[i] = solicitud;
                    break;
                }
            }
        }

        public void mostrarInformacion() {  no usages  new *
            System.out.println(this.toString());
        }

        @Override  new *
        public String toString() {
            return "Composicion{" +
                    "titulo='" + titulo + '\'' +
```

Composicion.java × Solicitud.java

```java
public class Composicion {  no usages  new *
        public void registrarSolicitud(Solicitud solicitud) {  no usages  new *
            }
        }

        public void mostrarInformacion() {  no usages  new *
            System.out.println(this.toString());
        }

        @Override  new *
        public String toString() {
            return "Composicion{" +
                    "titulo='" + titulo + '\'' +
                    ", duracionMinutos=" + duracionMinutos +
                    ", duracionSegundos=" + duracionSegundos +
                    ", generoMusical='" + generoMusical + '\'' +
                    ", fechaRegistro=" + fechaRegistro +
                    ", fechaEstreno=" + fechaEstreno +
                    '}';
        }
}
```

Código clase solicitud:

```java
package problema1;

import java.time.LocalDate;

public class Solicitud {  3 usages  new *

        private String nombreInterprete;  2 usages
        private LocalDate fechaSolicitud;  2 usages
        private LocalDate fechaAutorizacion;  2 usages
        private String companiaGrabadora;  2 usages

        public Solicitud(String nombreInterprete, LocalDate fechaSolicitud, String companiaGrabadora) {  no usages  new *
            this.nombreInterprete = nombreInterprete;
            this.fechaSolicitud = fechaSolicitud;
            this.companiaGrabadora = companiaGrabadora;
        }

        public void autorizar(LocalDate fechaAutorizacion) {  no usages  new *
            this.fechaAutorizacion = fechaAutorizacion;
        }

        public void mostrarSolicitud() {  no usages  new *
            System.out.println(this.toString());
        }
```

```java
    public class Solicitud {  3 usages  new *
        }

        public void autorizar(LocalDate fechaAutorizacion) {  no usages  new *
            this.fechaAutorizacion = fechaAutorizacion;
        }

        public void mostrarSolicitud() {  no usages  new *
            System.out.println(this.toString());
        }

        @Override  new *
        public String toString() {
            return "Solicitud{" +
                    "nombreInterprete='" + nombreInterprete + '\'' +
                    ", fechaSolicitud=" + fechaSolicitud +
                    ", fechaAutorizacion=" + fechaAutorizacion +
                    ", companiaGrabadora='" + companiaGrabadora + '\'' +
                    '}';
        }
    }
```

Prueba de código:

```
jshell> import java.time.LocalDate;

jshell>

jshell> Composicion c1=new Composicion ("Melodia Azul",3,45,"Rock", LocalDate.of(2024,5,10),LocalDate.of(2024,6,1));
c1 ==> Composicion{titulo='Melodia Azul', duracionMinuto ... , fechaEstreno=2024-06-01}

jshell> import problema1.Solicitud;

jshell> import java.time.LocalDate;

jshell> Solicitud s1=new Solicitud("Carlos Ruiz",LocalDate.of(2024,1,15),"Sonyc Music");
s1 ==> Solicitud{nombreInterprete='Carlos Ruiz', fechaSo ... iaGrabadora='Sonyc Music'}

jshell> import problema1.Composicion;

jshell> c1.agregarInterprete("Juan");

jshell> c1.registrarSolicitud(s1);

jshell> import problema1.Solicitud;
```

```
jshell> import problema1.Composicion;

jshell> c1
c1 ==> Composicion{titulo='Melodia Azul', duracionMinutos=3, duracionSegundos=45, generoMusical='Rock', fechaRegistro=2024-05-10, fechaEstreno=2024-06-01}
```

```
jshell> s1.autorizar(LocalDate.of(2024,1,20));

jshell> s1
s1 ==> Solicitud{nombreInterprete='Carlos Ruiz', fechaSolicitud=2024-01-15, fechaAutorizacion=2024-01-20, companiaGrabadora='Sonyc Music'}
```

Análisis del código

Al momento de correr el código importamos la clase de música y después agregamos la música, que va primero el nombre de la música, después el tiempo de la música, el genero y la fecha solicitada. Después importamos la clase solicitud y metimos el nombre del interprete y después agregamos la fecha. Agregamos la canción (c1) a el método agregarInterprete y tambien la agregamos a registrarSolicitud. Mandamos a llamar las variables para que aparezca es el toSrint().

Problema 2:

En cierta clínica están reorganizando los expedientes de los pacientes, de cada uno de ellos los ubican con su nombre, rfc, fecha de nacimiento, de cada consulta que se les ha realizado se tiene su fecha, síntomas (que es una lista de los síntomas que tiene incluyendo temperatura peso y estatura que se tomó), diagnóstico médico que se trata de una lista del (los) padecimiento(s) y la receta que consiste de una lista de medicamentos y recomendaciones del tratamiento que debe llevar a cabo. De un paciente se puede agregar consultas llevadas a cabo, calcular y obtener su edad.

Análisis del problema: el problema nos pide de cada paciente tener su nombre, rfc, fecha de nacimiento, de cada consulta que tenga el paciente, se tiene registrada la fecha, sus síntomas (en una lista de los síntomas), diagnostico medico que será una lista de los procedimientos que se le hicieron, la receta que consiste en la lista de medicamentos, a cada paciente nos pide poder agregarle consultas llevadas a cabo y calcular su edad con su fecha de nacimiento.

Clase consulta:

consulta			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
fecha	LocalDate	1	si
síntomas	String	1	si
peso	double	1	si
estatura	double	1	si
diagnosticoMedico	String	1	si
receta	String	1	si
temperatura	double	1	si

Operaciones consulta:

Consulta			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
mostrarConsulta	void		public

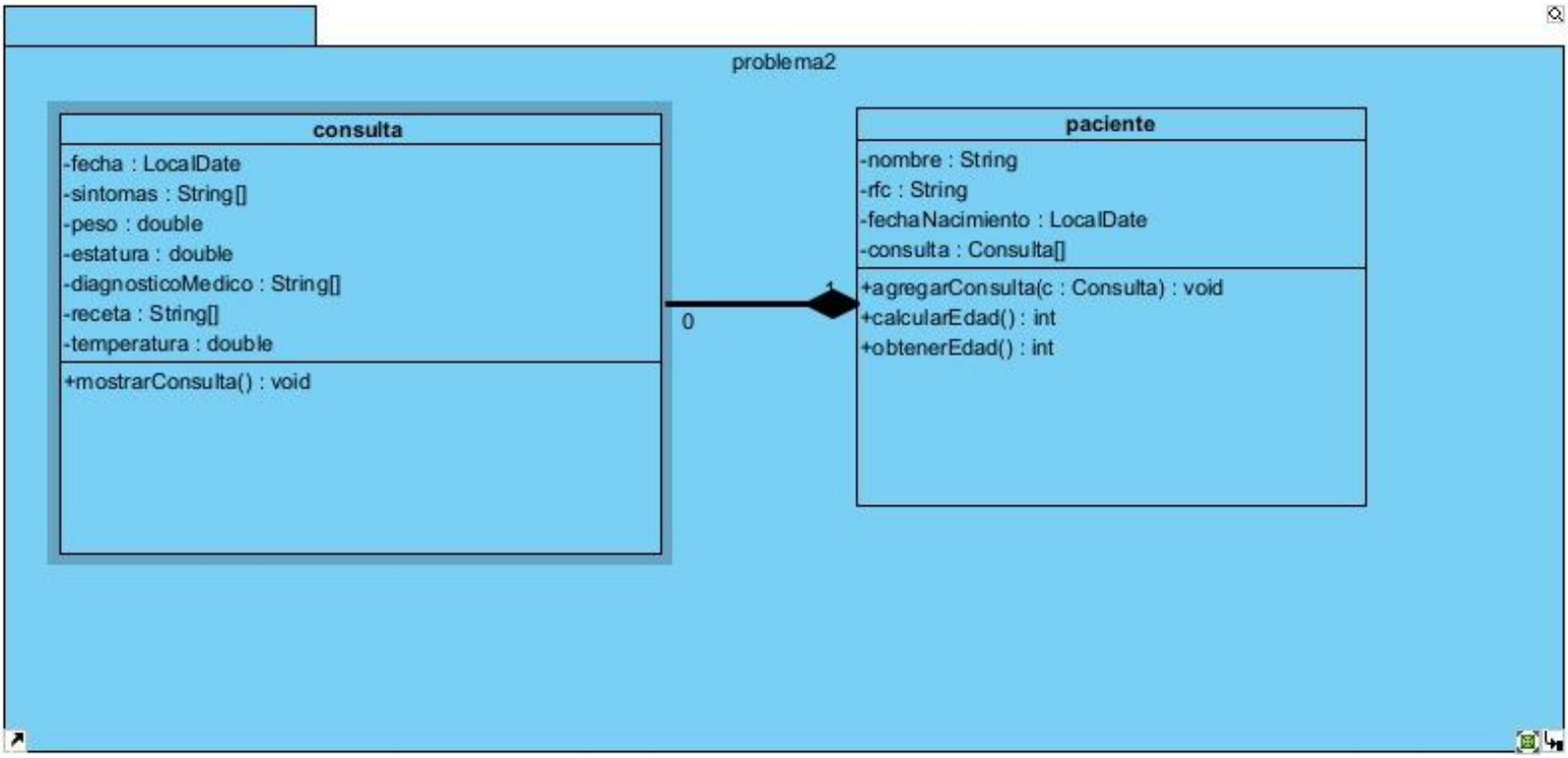
Clase paciente:

paciente			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
nombre	String	1	si
rfc	String	1	si
fechaNacimietno	LocalDate	1	si
consulta	consulta	1	si

Operaciones paciente:

paciente			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
agregarConsulta	void	consulta	public
calcularEdad	int		public
obtenerEdad	int		public

Diagrama:



Código clase consulta:

Paciente.java Consulta.java

```java
package problema2;

import java.time.LocalDate;

public class Consulta {  3 usages  new *
        private LocalDate fecha;  2 usages
        private String[] sintomas;  1 usage
        private double peso;  2 usages
        private double estatura;  2 usages
        private String[] diagnosticoMedico;  1 usage
        private String[] receta;  1 usage
        private double temperatura;  2 usages

        public Consulta(LocalDate fecha, double peso, double estatura, double temperatura) {  no usages  new *
            this.fecha = fecha;
            this.peso = peso;
            this.estatura = estatura;
            this.temperatura = temperatura;

            sintomas = new String[10];
            diagnosticoMedico = new String[10];
            receta = new String[10];
        }
```

```java
Paciente.java    Consulta.java ×

public class Consulta {  3 usages  new *
    public Consulta(LocalDate fecha, double peso, double estatura, double temperatura) {  no usages  new *
        diagnosticoMedico = new String[10];
        receta = new String[10];
    }

    public void mostrarConsulta() {  no usages  new *
        System.out.println(this.toString());
    }

    @Override  new *
    public String toString() {
        return "Consulta{" +
                "fecha=" + fecha +
                ", peso=" + peso +
                ", estatura=" + estatura +
                ", temperatura=" + temperatura +
                '}';
    }
}
```

Codigo clase Paciente:

```java
Paciente.java ×    Consulta.java

package problema2;

import java.time.LocalDate;
import java.time.Period;

public class Paciente {  no usages  new *
        private String nombre;  2 usages
        private String rfc;  2 usages
        private LocalDate fechaNacimiento;  3 usages
        private Consulta[] consultas;  4 usages

        public Paciente(String nombre, String rfc, LocalDate fechaNacimiento) {  no usages  new *
            this.nombre = nombre;
            this.rfc = rfc;
            this.fechaNacimiento = fechaNacimiento;
            consultas = new Consulta[20];
        }

        public void agregarConsulta(Consulta c) {   // CORREGIDO  no usages  new *
            for (int i = 0; i < consultas.length; i++) {
                if (consultas[i] == null) {
                    consultas[i] = c;
                    break;
                }
            }
        }
```

```java
Paciente.java ×   Consulta.java

public class Paciente {  no usages  new *

        public int calcularEdad() {  2 usages  new *
            return Period.between(fechaNacimiento, LocalDate.now()).getYears();
        }

        public int obtenerEdad() {  no usages  new *
            return calcularEdad();
        }

        @Override  new *
        public String toString() {
            return "Paciente{" +
                    "nombre='" + nombre + '\'' +
                    ", rfc='" + rfc + '\'' +
                    ", fechaNacimiento=" + fechaNacimiento +
                    ", edad=" + calcularEdad() +
                    '}';
        }
}
```

Prueba codigo:

```
PS C:\Users\Fernando\OneDrive\Documents\intellij Idea\actividad3> jshell --class-path out/production/actividad3
|  Welcome to JShell -- Version 24.0.2
|  For an introduction type: /help intro

jshell> import problema2.Paciente;

jshell> import java.time.LocalDate;

jshell> Paciente p1=new Paciente ("Eliel Adrian", "CAHE071203H2C941", LocalDate.of(2007,12,03));
p1 ==> Paciente{nombre='Eliel Adrian', rfc='CAHE071203H2 ... iento=2007-12-03, edad=18}

jshell> import problema2.Consulta;

jshell> import java.time.LocalDate;

jshell> Consulta c1=new Consulta(LocalDate.of(2026,01,24),70,1.68,37.5);
c1 ==> Consulta{fecha=2026-01-24, peso=70.0, estatura=1.68, temperatura=37.5}

jshell> import problema2.Paciente;

jshell> p1.agregarConsulta(c1);

jshell> p1.calcularEdad();
$9 ==> 18

jshell> p1
p1 ==> Paciente{nombre='Eliel Adrian', rfc='CAHE071203H2C941', fechaNacimiento=2007-12-03, edad=18}
```

Análisis Del Código:

Agregamos en la clase paciente un paciente y mandamos los datos que pedían que fueron nombre, RFC y fecha de nacimiento, después importamos la clase consulta y metimos la consulta del paciente y agregamos al paciente al método de agregarconsulta que se le manda la consulta (c1) y mandamos al paciente al método calcularEdad y mostramos al final al paciente(p1) con el método toString()

Problema 3:

Una línea de transporte de carga desea sistematizar sus transportes que utiliza, de cada vehículo se tiene un registro de marca, modelo, cantidad máxima de carga permitido, fecha de adquisición y una lista de todas los viajes realizados. De cada viaje realizado, se tiene ciudad destino, dirección, fecha de viaje, fecha de regreso, descripción de la carga y monto del viaje. Cada viaje que le contratan a la línea de transporte se asigna a un vehículo acorde a la carga que se transportará. Si un viaje es cancelado por el cliente el viaje se dará de baja del vehículo al cual se le asignó.

Análisis del problema: el problema nos pide de cada transporte tener registrado la marca, modelo, cantidad máxima de carga permitida, fecha de adquisición, y una lista de sus viajes realizados, nos pide que de cada viaje realizado se tiene registrada la ciudad de destino, dirección, fecha de viaje y fecha de regreso, descripción de la carga y el monto de viaje, de cada viaje que se contrata se asigna un vehículo acorde a la carga, y se podrá cancelar el viaje si el cliente así lo desea.

Clase transporte:

transporte			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
Marca	String	1	si
modelo	String	1	si
cantidadMaxima	double	1	si
fechaAdquisicion	LocalDate	1	si
viajesRealizados	viajeRealizado	1	si

Operaciones transporte:

transporte			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
asignarViaje	void	viajeRealizado	public
cancelarViaje	void	viajeRealizado	public
mostrarViajes	void		public

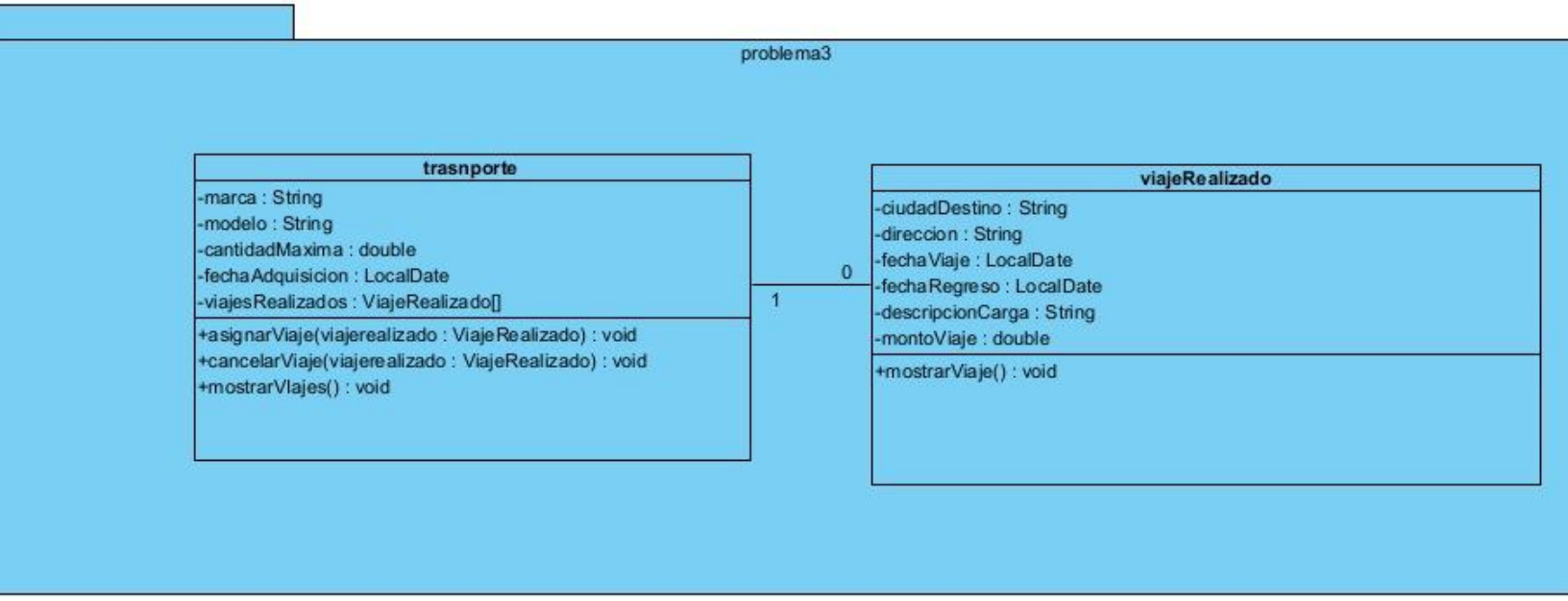
Clase viaje realizado:

Viaje Realizado			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
ciudadDestino	String	1	si
direccion	String	1	si
fechaViaje	LocalDate	1	si
fechaRegistro	LocalDate	1	si
descripcionCarga	String	1	si
montoViaje	double	1	si

Operaciones viaje realizado:

Viaje realizado			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
mostrarViaje	void		public

Diagrama:



Código clase transporte:

Transporte.java × ViajeRealizado.java

```java
package problema3;

import java.time.LocalDate;

public class Transporte {  no usages  new *
        private String marca;  2 usages
        private String modelo;  2 usages
        private double cantidadMaxima;  2 usages
        private LocalDate fechaAdquisicion;  2 usages
        private ViajeRealizado[] viajesRealizados;  8 usages

        public Transporte(String marca, String modelo, double cantidadMaxima, LocalDate fechaAdquisicion) {  no usages

            this.marca = marca;
            this.modelo = modelo;
            this.cantidadMaxima = cantidadMaxima;
            this.fechaAdquisicion = fechaAdquisicion;

            viajesRealizados = new ViajeRealizado[20];
        }
```

Transporte.java × ViajeRealizado.java

```java
public class Transporte {  no usages  new *

        public void asignarViaje(ViajeRealizado viajeRealizado) {  no usages  new *
            for (int i = 0; i < viajesRealizados.length; i++) {
                if (viajesRealizados[i] == null) {
                    viajesRealizados[i] = viajeRealizado;
                    break;
                }
            }
        }

        public void cancelarViaje(ViajeRealizado viajeRealizado) {  no usages  new *
            for (int i = 0; i < viajesRealizados.length; i++) {
                if (viajesRealizados[i] == viajeRealizado) {
                    viajesRealizados[i] = null;
                    break;
                }
            }
        }

        public void mostrarViajes() {  no usages  new *
            for (ViajeRealizado v : viajesRealizados) {
                if (v != null) {
                    System.out.println(v);
                }
            }
        }
```

```java
Transporte.java ×   ViajeRealizado.java

public class Transporte {  no usages  new *

        public void mostrarViajes() {  no usages  new *
            for (ViajeRealizado v : viajesRealizados) {
                if (v != null) {
                    System.out.println(v);
                }
            }
        }

        @Override  new *
        public String toString() {
            return "Transporte{" +
                    "marca='" + marca + '\'' +
                    ", modelo='" + modelo + '\'' +
                    ", cantidadMaxima=" + cantidadMaxima +
                    ", fechaAdquisicion=" + fechaAdquisicion +
                    '}';
        }
    }
```

Código Clase viajeRealizado:

```java
Transporte.java ×   ViajeRealizado.java ×

package problema3;

import java.time.LocalDate;

public class ViajeRealizado {  5 usages  new *
        private String ciudadDestino;  2 usages
        private String direccion;  2 usages
        private LocalDate fechaViaje;  2 usages
        private LocalDate fechaRegreso;  2 usages
        private String descripcionCarga;  2 usages
        private double montoViaje;  2 usages

        public ViajeRealizado(String ciudadDestino, String direccion, LocalDate fechaViaje,  no usages  new *
                              LocalDate fechaRegreso, String descripcionCarga, double montoViaje) {

            this.ciudadDestino = ciudadDestino;
            this.direccion = direccion;
            this.fechaViaje = fechaViaje;
            this.fechaRegreso = fechaRegreso;
            this.descripcionCarga = descripcionCarga;
            this.montoViaje = montoViaje;
        }

        public void mostrarViaje() {  no usages  new *
            System.out.println(this.toString());
        }
```

```java
Transporte.java    ViajeRealizado.java ×
public class ViajeRealizado { 5 usages new *
        }

        public void mostrarViaje() { no usages new *
            System.out.println(this.toString());
        }

        @Override new *
        public String toString() {
            return "ViajeRealizado{" +
                    "ciudadDestino='" + ciudadDestino + '\'' +
                    ", direccion='" + direccion + '\'' +
                    ", fechaViaje=" + fechaViaje +
                    ", fechaRegreso=" + fechaRegreso +
                    ", descripcionCarga='" + descripcionCarga + '\'' +
                    ", montoViaje=" + montoViaje +
                    '}';
        }
}
```

Prueba Código:

```
jshell> import
jshell> import problema3.Transporte;

jshell> import java.time.LocalDate;

jshell> Transporte t1=new Transporte("Volvo","FH16",20000,LocalDate.of(2020,5,5));
t1 ==> Transporte{marca='Volvo', modelo='FH16', cantidad ... chaAdquisicion=2020-05-05}

jshell> import problema3.ViajeRealizado;

jshell> import java.time.LocalDate;

jshell> ViajeRealizado v1=new ViajeRealizado ("Veracruz", "Av industrial 120",LocalDate.of(2026,4,1),LocalDate.of(2026,4,3),"Electrodomesticos",15000);
v1 ==> ViajeRealizado{ciudadDestino='Veracruz', direccio ... icos', montoViaje=15000.0}

jshell> import problema3.Transporte;

jshell> t1.asignarViaje(v1);

jshell> t1.mostrarViajes();
ViajeRealizado{ciudadDestino='Veracruz', direccion='Av industrial 120', fechaViaje=2026-04-01, fechaRegreso=2026-04-03, descripcionCarga='Electrodomesticos',
montoViaje=15000.0}

jshell> t1
t1 ==> Transporte{marca='Volvo', modelo='FH16', cantidadMaxima=20000.0, fechaAdquisicion=2020-05-05}
```

Análisis del Código:

Importamos la clase de transporte y agregamos el transporte que se meten los datos que piden y después se importa la clase viajesRealizados y mandamos el viaje que se va a realizar y después mandamos el transporte(t1) al método de asignarViaje y lo mandamos también a mostrarViajes y en ese mismo nos sale el meotodo toString()

Conclusiones:

En esta actividad pudimos reforzar nuestros conocimientos sobre la abstracción y el encapsulamiento de clases, ya que hubo errores que nos costaron un poco de tiempo poder encontrar la solución por algunos métodos, esta actividad nos enseño a poder interpretar de maneras distintas la información presentada, ya que se sintió un poco diferente a las anteriores ya que planteaba cosas un poco más específicas, y con algunas listas de datos requeridos pero pudimos identificarlos para poder dalre solución a los problemas planteados.

URl GitHub:

https://github.com/irvingemanuel2003-a11y/actividad3.git

https://github.com/soysantiagorb/Actividad3